

Table of Contents

1. Bootstrap Packs	1
1.1. What a BootstrapPack is	1
1.2. Why BootstrapPacks exist	1
1.3. BootstrapPacks vs seed packs	2
1.4. When to use one vs the other	2
1.5. Rule of thumb	3
1.6. Step kinds	3
1.7. Apply modes and policies	3
1.8. SPI model	4
1.9. Admin REST API	5
1.10. Startup application	5
1.11. Run history	6
1.12. Examples	6
1.13. Choosing the right abstraction	7
1.14. See also	7

1. Bootstrap Packs

BootstrapPacks are the framework mechanism for idempotent runtime setup work that is bigger than declarative seeding.

They are intentionally distinct from migrations and seed packs:

- migrations change database structure and initialization state
- seed packs materialize deterministic datasets
- BootstrapPacks orchestrate operational setup steps across one or more systems

`quantum-bootstrap-core` provides the BootstrapPack runtime, startup runner, models, and SPI hooks. Product modules contribute actual packs and step handlers by depending on that endpoint-free artifact.

`quantum-bootstrap-rest` is the opt-in system-management adapter that exposes the admin API. It is selected by `quantum-system-service`; ordinary application backends should omit it and consume the generated system-service SDK instead.

1.1. What a BootstrapPack is

A BootstrapPack is a named, versioned sequence of steps:

- `packRef`
- `packVersion`
- `productRef`
- `profileRef`
- ordered `steps`

Each step has:

- `stepRef`
- `description`
- `kind`
- `applyPolicy`
- `dependsOn`
- `config`

At runtime, `BootstrapPackService` resolves a pack from the registry, builds a scope, executes each step through the matching `BootstrapPackStepHandler`, and records a `BootstrapPackRun`.

1.2. Why BootstrapPacks exist

Seed packs are excellent for baseline data, but some startup and tenant-activation work is not just "load these JSON records":

- calling a product service to ensure a resource exists
- triggering an HTTP integration to an external platform
- validating that a prerequisite already exists
- publishing workflow or solution metadata
- sequencing several actions with dependencies and run history

That is the gap BootstrapPacks fill.

1.3. BootstrapPacks vs seed packs

Question	Seed packs	BootstrapPacks
Primary job	Load deterministic baseline data	Coordinate idempotent operational setup steps
Input format	Manifest plus JSON/NDJSON datasets	Java <code>BootstrapPackDefinition</code> plus step configs
Best for	Code lists, policies, defaults, reference rows, baseline records	Calling services, verifying prerequisites, external side effects, multi-step orchestration
Idempotency model	Dataset checksums plus natural keys	Step handler logic plus apply mode and apply policy
Cross-system work	Poor fit	First-class use case
Dependencies	Includes/archetypes inside manifests	<code>dependsOn</code> between steps inside one pack
Typical owner	Framework or product module shipping data	Product/application module shipping setup logic

1.4. When to use one vs the other

Use a seed pack when the desired result is "these records should exist in the tenant realm."

Examples:

- install code lists
- create default policies
- load default feature metadata
- establish baseline survey definitions

Use a BootstrapPack when the desired result is "run these steps to make the environment ready."

Examples:

- ensure an app service has provisioned a workspace
- call an external API to register a tenant

- run assertions before enabling a solution
- publish a workflow package after required seed data exists

Use both together when needed:

- a BootstrapPack step can depend on a **SEED_PACK** step to materialize baseline data first
- later steps can then call services or external APIs that rely on that seeded data

1.5. Rule of thumb

If you can review the change entirely as data files, prefer a seed pack.

If the change needs code, orchestration, side effects, or conditional runtime checks, prefer a BootstrapPack.

1.6. Step kinds

The framework model currently defines these step kinds:

- **SEED_PACK**
- **APP_SERVICE**
- **HTTP_CALL**
- **ASSERTION**
- **WORKFLOW_PLATFORM_PUBLICATION**

The enum is shared infrastructure. Actual behavior depends on the **BootstrapPackStepHandler** implementations contributed by the application or product modules on the classpath.

1.7. Apply modes and policies

BootstrapPack apply mode controls how the pack is being run:

- **APPLY_MISSING**
- **REAPPLY**
- **VALIDATE_ONLY**

Step apply policy describes the intended handler behavior:

- **ONCE_PER_SCOPE**
- **WHEN_MISSING**
- **ALWAYS_IDEMPOTENT**

The framework passes both values into handlers through **BootstrapStepRequest**. Handler authors decide what those semantics mean for a specific step kind.

1.8. SPI model

Applications contribute packs with `BootstrapPackContributor`:

```
@ApplicationScoped
public class DemoBootstrapContributor implements BootstrapPackContributor {
    @Override
    public Collection<BootstrapPackDefinition> bootstrapPacks() {
        return List.of(new BootstrapPackDefinition(
            "demo-pack",
            "2026.04.0",
            "b2bi",
            "local-demo",
            List.of(
                new BootstrapPackStepDefinition(
                    "ensure-a",
                    "Ensure resource A exists",
                    BootstrapStepKind.APP_SERVICE,
                    BootstrapStepApplyPolicy.WHEN_MISSING,
                    List.of(),
                    Map.of("resource", "A")
                ),
                new BootstrapPackStepDefinition(
                    "ensure-b",
                    "Ensure resource B exists",
                    BootstrapStepKind.APP_SERVICE,
                    BootstrapStepApplyPolicy.WHEN_MISSING,
                    List.of("ensure-a"),
                    Map.of("resource", "B")
                )
            )
        ));
    }
}
```

Handlers implement the work for one or more step kinds:

```
@ApplicationScoped
public class DemoAppServiceHandler implements BootstrapPackStepHandler {
    @Override
    public boolean supports(BootstrapStepKind kind) {
        return kind == BootstrapStepKind.APP_SERVICE;
    }

    @Override
    public BootstrapStepResult execute(BootstrapStepRequest request) {
        return new BootstrapStepResult(
            BootstrapStepStatus.COMPLETED,
            BootstrapStepOutcome.CREATED,

```

```

        Map.of("handledStep", request.stepRef())
    );
}
}

```

1.9. Admin REST API

The system-management service exposes the `quantum-bootstrap-rest` admin endpoints at `/admin/bootstrap-packs`:

Endpoint	Purpose
GET <code>/admin/bootstrap-packs</code>	List available pack definitions
GET <code>/admin/bootstrap-packs/pending/{realm}</code>	List packs with no successful completed run for the realm
GET <code>/admin/bootstrap-packs/history/{realm}</code>	Show recorded runs for the realm
POST <code>/admin/bootstrap-packs/validate/{realm}</code>	Validate all selected packs without applying changes
POST <code>/admin/bootstrap-packs/{realm}/{packRef}/validate</code>	Validate one pack
POST <code>/admin/bootstrap-packs/apply/{realm}</code>	Apply all selected packs
POST <code>/admin/bootstrap-packs/{realm}/{packRef}/apply</code>	Apply one pack

Useful query parameters:

- `filter=pack-a,pack-b`
- `mode=APPLY_MISSING`
- `mode=REAPPLY`
- `mode=VALIDATE_ONLY`

These endpoints are protected for `admin` and `system` roles.

1.10. Startup application

BootstrapPacks can also be auto-applied at startup by `BootstrapPackStartupRunner`.

Relevant properties:

```

quantum.bootstrap-pack.enabled=true
quantum.bootstrap-pack.apply.on-startup=false
quantum.bootstrap-pack.apply.realms=system-com,mycompany-com,test-com

```

```
quantum.bootstrap-pack.apply.filter=pack-a,pack-b
```

If startup application is enabled, the runner:

1. resolves the target realms
2. resolves the selected packs
3. runs each pack as `APPLY_MISSING`
4. logs failures and pending packs

1.11. Run history

The framework records `BootstrapPackRun` objects, including:

- pack ref and version
- execution scope such as realm and environment
- per-step status and outcome
- start and finish timestamps

By default, the framework uses an in-memory run repository. That is useful for local development and tests, but products that need durable operational history should plug in a persistent `BootstrapPackRunRepository`.

1.12. Examples

Example 1: Use a seed pack only

Requirement: Add baseline code lists, policies, and default records for every new tenant.

Best choice: Use a seed pack.

Reason: The desired result is purely deterministic data in MongoDB.

Example 2: Use a `BootstrapPack` only

Requirement: After a tenant is created, call a product service to create a workspace and then verify it exists.

Best choice: Use a `BootstrapPack` with `APP_SERVICE` and `ASSERTION` steps.

Reason: The work is procedural and depends on service logic, not just document inserts.

Example 3: Use both together

Requirement: Seed workflow definitions first, then publish them to an external workflow platform.

Best choice:

- Use a seed pack for the workflow and policy records

- Use a BootstrapPack with a **SEED_PACK** step followed by **WORKFLOW_PLATFORM_PUBLICATION**

Reason: The first half is declarative data, while the second half is an operational publication step.

1.13. Choosing the right abstraction

Prefer seed packs by default for tenant baseline data.

Move to BootstrapPacks when:

- the setup spans more than one system
- steps must run in order with dependencies
- you need dry-run validation semantics
- the setup is product-owned logic rather than framework-owned baseline data

1.14. See also

- [Seed packs and declarative tenant seeding](#)
- [Email Templates](#)
- [Tenant onboarding and access invites](#)